

Advanced Parallel Programming & Architecture

Laboratory Examination: Empirical Microarchitectural Bottlenecks

Student Name: Symbat Aldanova Student ID: 230103170 Instructor: Sufyan bin Uzayr Laptop:MacBook Pro
Processor: Apple M4 Physical Cores: 10 (4 Performance + 6 Efficiency) Logical Cores: 10 RAM: 16 GBL1 Data
Cache: 64 KB (reported) L2 Cache: 4 MB (reported) Operating System: macOS Python Version: 3.12.7

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Experimental Results

Worker Cores	Time (s)	Speedup	Ideal Speedup	Efficiency
1	1.2398	1.00x	1.00x	100%
2	0.6372	1.95x	2.00x	97.5%
4	0.3437	3.62x	4.00x	90.5%
8	0.2847	4.37x	8.00x	54.6%
12	0.2904	4.28x	12.00x	35.7%
16	0.3058	4.07x	16.00x	25.4%

The independently measured single-process baseline was $T(1) = 1.2431$ seconds.

Q1.1. Identify the inflection point.

The observed speedup reaches its maximum at 8 workers, with a speedup of 4.37x.

After 8 workers, performance starts to decrease.

My Apple M4 processor has 10 physical cores: 4 Performance cores and 6 Efficiency cores.

The processor does not use SMT.

The measured inflection point is $p^* = 8$. Increasing the worker count beyond this point introduces additional scheduling overhead and resource contention.

Q1.2. Calculate the parallel fraction P.

Using the experimental results:

$$T(1) = 1.2431 \text{ s} \quad T(4) = 0.3437 \text{ s}$$

Observed speedup:

$$S(4) = 1.2431 / 0.3437 = 3.61798$$

According to Amdahl's Law:

$$S(4) = 1 / [(1 - P) + P/4]$$

Rearranging:

$$1 / S(4) = 1 - 3P/4$$

$$P = (4/3) \times (1 - 1/S(4))$$

$$P = (4/3) \times (1 - 1/3.61798)$$

$$P \approx 0.9648 = 96.48\%$$

This is the estimated parallel fraction based on the measured execution times.

Q1.3. Calculate the asymptotic speedup limit.

According to Amdahl's Law:

$$S_{\max} = 1 / (1 - P)$$

$$S_{\max} = 1 / (1 - 0.9648)$$

$$S_{\max} \approx 28.4x$$

This represents the theoretical maximum speedup under the estimated Amdahl model.

Even if a 128-core server were used, the serial portion of the workload would limit the achievable speedup.

Real-world performance would also depend on process overhead, memory bandwidth and communication costs.

TASK 2: CACHE COHERENCE & FALSE SHARING

Experimental Results

Configuration	Trial 1	Trial 2	Trial 3	Median
Adjacent Indices	0.4295	0.4335	0.4412	0.4335
Padded Indices	0.4225	0.4290	0.4271	0.4271

Slowdown Factor:

$$0.4335 / 0.4271 \approx 1.01x$$

The measured difference between the two configurations was approximately 1%.

Q2.1. Explain the memory footprint and padding.

On a 64-bit CPython implementation, a list stores references to Python objects.

Each reference occupies 8 bytes.

For a stride of 16 elements:

$$16 \times 8 = 128 \text{ bytes}$$

This is larger than a typical 64-byte cache line.

The padding separates the list reference slots accessed by different threads.

However, Python integers are separate objects, and their memory placement is not guaranteed to follow the list layout.

Therefore, this Python experiment does not guarantee that the integer objects themselves occupy separate cache lines.

Q2.2. Explain the MESI state transitions.

The MESI protocol maintains cache coherence between processor cores.

Its four states are Modified, Exclusive, Shared and Invalid.

When two cores write to different variables located in the same cache line, ownership of that cache line may move between the cores.

For example, Core 0 obtains exclusive ownership and modifies its data. If Core 1 subsequently requests ownership to write to the same cache line, Core 0 must relinquish ownership.

The coherence mechanism invalidates conflicting cached copies and transfers ownership.

Repeated ownership transfers can increase execution time.

In this experiment, the measured slowdown was only 1.01x. Because CPython uses the Global Interpreter Lock (GIL), these results do not establish that hardware-level false sharing occurred.

Q2.3. Describe two mechanisms to eliminate false sharing.

1. C++ alignas(64)

C++ provides the alignas specifier, which can align data structures to a specified memory boundary.

Aligning frequently modified variables and separating them into different cache lines can reduce false sharing.

2. Java @Contended

Java provides the @Contended annotation to reduce false sharing by adding padding around selected fields or classes.

It is commonly used in performance-sensitive concurrent applications, subject to JVM support and configuration.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Experimental Results

Architecture	Target Sum	Actual Sum	Time (s)
Unsafe Counter	2,000,000	2,000,000	0.0726
Locked Counter	2,000,000	2,000,000	0.1965
Lockless Counter	2,000,000	2,000,000	0.0276

Contention Cost Multiplier: 2.71x

Lockless Speedup: 7.13x

Q3.1. Analyze the Unsafe Accumulator output.

The expected result was 2,000,000.

The actual result was also 2,000,000.

Therefore, the observed data corruption count was zero.

However, this does not mean that an unprotected shared counter is universally safe.

A read-modify-write operation conceptually consists of three steps:

1. LOAD the current value.
2. ADD one to the value.
3. STORE the updated value.

In a concurrent execution, two threads may read the same original value and overwrite each other's updates.

This is known as a lost update.

In my CPython 3.12.7 experiment, the GIL and the execution behavior prevented observable lost updates during this run.

Q3.2. Implement a Lockless Thread-Local Accumulator.

The lockless implementation uses a private counter for each thread.

Each thread performs its calculations independently and stores its result in a separate position.

After all threads finish, the partial results are combined using `sum()`.

```
def bench_lockless():
    partial = [0] * NUM_THREADS
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work(index):
        local_count = 0
        for _ in range(ops_per_thread):
            local_count += 1
        partial[index] = local_count
    threads = [threading.Thread(target=work, args=(i,)) for i in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    total = sum(partial)
    elapsed = time.perf_counter() - start
    return total, elapsed
```

Experimental results:

Locked Counter: 0.1965 s Lockless Counter: 0.0276 s

Speedup = 0.1965 / 0.0276

Speedup $\approx 7.12x$ (7.13x reported by the program)

The lockless implementation produced the correct result of 2,000,000 and exceeded the required 2x speedup.

Q3.3. Explain why eliminating shared mutable state is superior.

Eliminating shared mutable state reduces the need for synchronization.

When each thread uses a private counter, threads do not compete for the same shared variable during accumulation.

This reduces lock contention and synchronization overhead.

The final reduction combines the partial results after the threads finish.

In my experiment, the lockless implementation was significantly faster than the locked implementation.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Experimental Results

Workers	Compute Time (s)	Compute Scaling	Memory Time (s)	Memory Degradation
1	0.5854	1.00x	0.2751	1.00x
2	0.6762	1.16x	0.3805	1.38x
4	0.8036	1.37x	0.8673	3.15x

Q4.1. Analyze the scaling divergence.

The compute-bound workload increased from 0.5854 seconds with one worker to 0.8036 seconds with four workers.

The memory-bound workload increased from 0.2751 seconds to 0.8673 seconds.

The memory-bound workload showed a much larger increase in execution time.

This is consistent with increased memory allocation, memory traffic and competition for memory bandwidth.

However, the benchmark increases the total amount of work as the worker count increases. It is therefore not a fixed-workload strong-scaling experiment.

The measurements are consistent with memory resource contention, but they do not independently prove that the DRAM bus reached its maximum bandwidth.

Q4.2. Investigate the physical memory specifications.

My laptop is a MacBook Pro with an Apple M4 processor and 16 GB of unified memory.

Apple Silicon uses a unified memory architecture in which the CPU and GPU share the same memory system.

The memory bandwidth specification and the exact memory-channel configuration were not independently verified in this experiment.

The available memory bandwidth limits how quickly processor cores can access data.

When several cores request large amounts of data simultaneously, they compete for the available bandwidth.

Adding more cores cannot increase performance indefinitely when memory bandwidth becomes the limiting resource.

Q4.3. Compare CUDA GPU kernels and CPU OpenMP.

For a memory-bandwidth-saturated embedding pipeline, the choice depends on the hardware and the amount of data transferred.

A GPU with high memory bandwidth may improve performance when the workload is suitable for parallel execution.

However, transferring data between CPU and GPU memory may introduce additional overhead.

CPU OpenMP can reduce some scheduling overhead and improve parallel execution, but adding threads cannot eliminate a physical memory-bandwidth limitation.

My Apple M4 has an integrated GPU and unified memory. CUDA is not supported on this hardware.

For this machine, a CPU implementation using efficient parallelism and memory access patterns is a practical approach.

A GPU implementation using a supported framework, such as Metal, could be evaluated separately.

CONCLUSION

This laboratory investigated parallel execution, cache coherence, synchronization overhead and memory performance using real measurements on an Apple M4 MacBook Pro.

Task 1 demonstrated that increasing the number of worker processes improved performance up to eight workers. Beyond this point, the measured speedup decreased.

Task 2 showed a small difference between adjacent and padded memory layouts. The experiment also demonstrated the limitations of using CPython threads to investigate hardware-level false sharing.

Task 3 demonstrated that thread-local accumulation can reduce synchronization overhead. The lockless implementation achieved the correct result and was approximately 7.13 times faster than the locked implementation.

Task 4 demonstrated different performance behavior for compute-intensive and memory-intensive workloads as the number of worker processes increased.

Overall, the experiments show that parallel performance depends on hardware architecture, synchronization, memory access patterns and the overhead of managing parallel workers.